



Programação Avançada 2025-26

[15] Padrões de Software | State

Patrícia Macedo & Bruno Silva

Sumário

- Padrão **State**
 - Enquadramento
 - Motivação
 - Solução Proposta (pelo padrão)
 - Prós e contras
 - Exemplo de Aplicação
 - Exercícios
 - Prós e contras

Enquadramento

- O padrão **state** é um padrão de desenho *comportamental*.



Padrão Comportamental

Estes padrões incidem sobre algoritmos e na atribuição/divisão de responsabilidades entre classes.

State

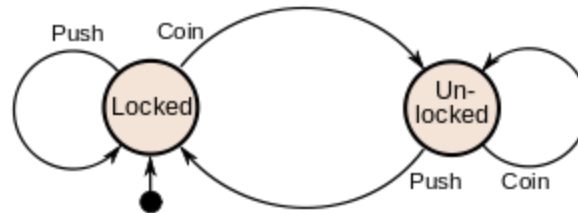
- Proposto em *Design Patterns: Elements of Reusable Object-Oriented Software* (1994)

THE 23 GANG OF FOUR DESIGN PATTERNS

 Abstract Factory	 Facade	 Proxy
 Adapter	 Factory Method	 Observer
 Bridge	 Flyweight	 Singleton
 Builder	 Interpreter	 State
 Chain of Responsibility	 Iterator	 Strategy
 Command	 Mediator	 Template Method
 Composite	 Memento	 Visitor
 Decorator	 Prototype	

State

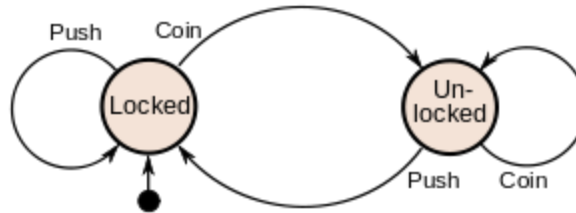
Está diretamente relacionado com *máquinas de estados finitos*.



Motivação

Permite que um objeto altere o seu comportamento, mediante o estado em que se encontra.

State



- Existe um número **finito** de *estados* nos quais um *objeto* se pode encontrar;
- Dentro de cada *estado* o objeto comporta-se de forma diferente;
- Certos *inputs* podem fazer o objeto *transitar* de *estado*, ou não;
- As *regras de transição* são também *finitas* e *pre-determinadas*.

State

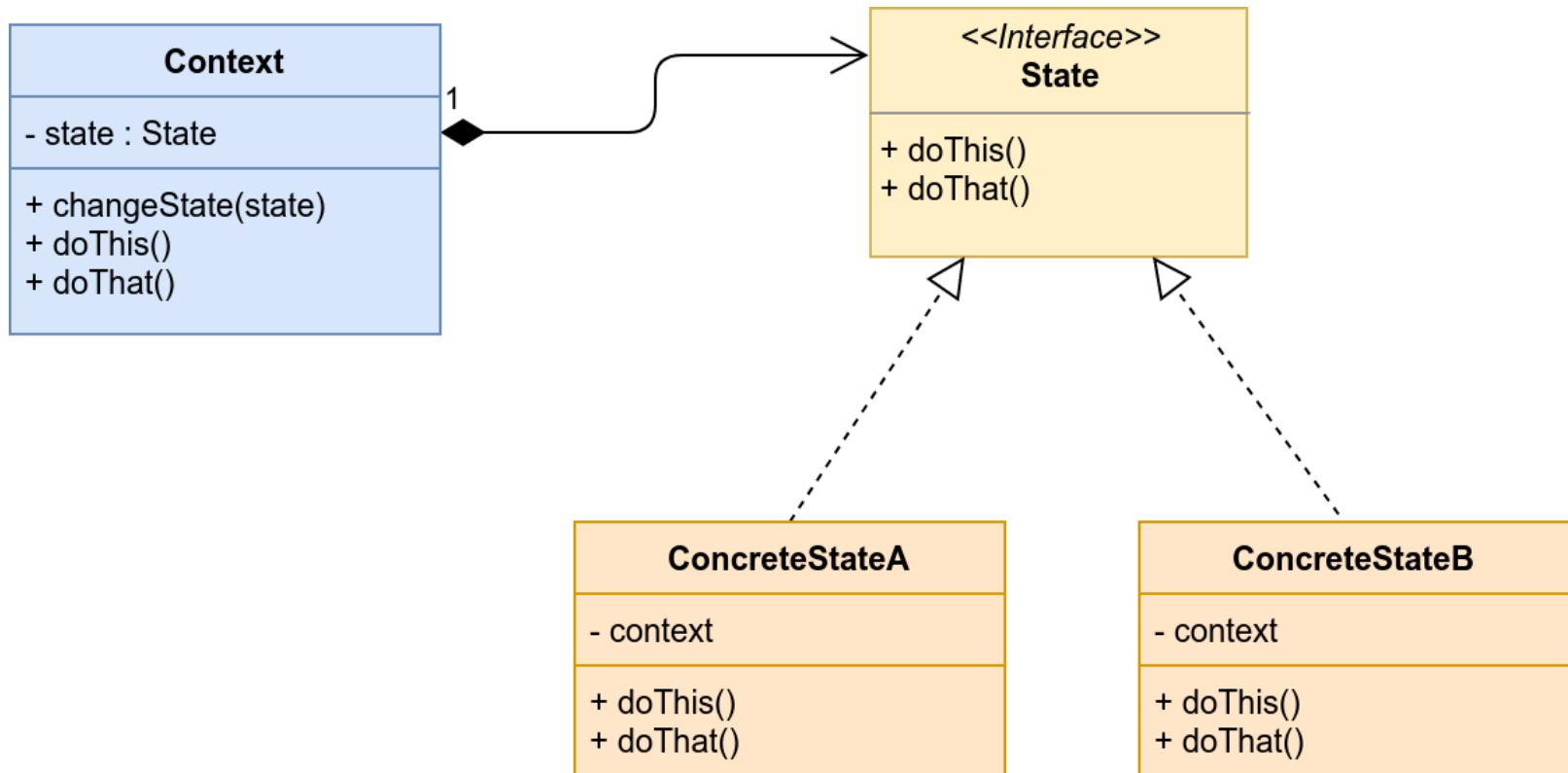
Permite evitar estruturas condicionais difíceis de manter em máquina de estados mais complexas. Para o exemplo anterior (simples) teríamos, e.g.:

```
public class Machine {
    enum State {NOCOIN, HASCOIN, BREWING};
    private State state;
    public Machine() { state = State.NOCOIN; }
    public void insertCoin() {
        if(state == State.NOCOIN) {
            //accept coin and change state
            state = state.HASCOIN;
        } else if(state == State.HASCOIN..) {
            //return coin and mantain state
        } else if(state == State.BREWING) ...
    }
}
```

💡 Se forem adicionados novos *estados* e *regras de transição* será difícil manter as estruturas condicionais.

State

Diagrama de classes da **solução proposta** pelo padrão:



State

Participantes e responsabilidades:

- **Context:** Guarda uma referência para um dos estados concretos e delega nele todo o comportamento específico do estado atual. Permite que o seu estado seja alterado.
- **State:** Declara os métodos específicos de um estado. Estes métodos serão "comuns" a todos os estados concretos, variando a sua implementação.
- **Concrete states:** Representam um estado possível e implementam os métodos de *State*, refletindo o comportamento do objeto para esse estado.

Problema real: 🌍 Coffe Machine



O seguinte repositório contém a aplicação do padrão **state** relativo ao problema de imulação do funcionamento de uma máquina de venda de cafes.

<https://github.com/estsetubal-pa/StateCoffeeMachine>

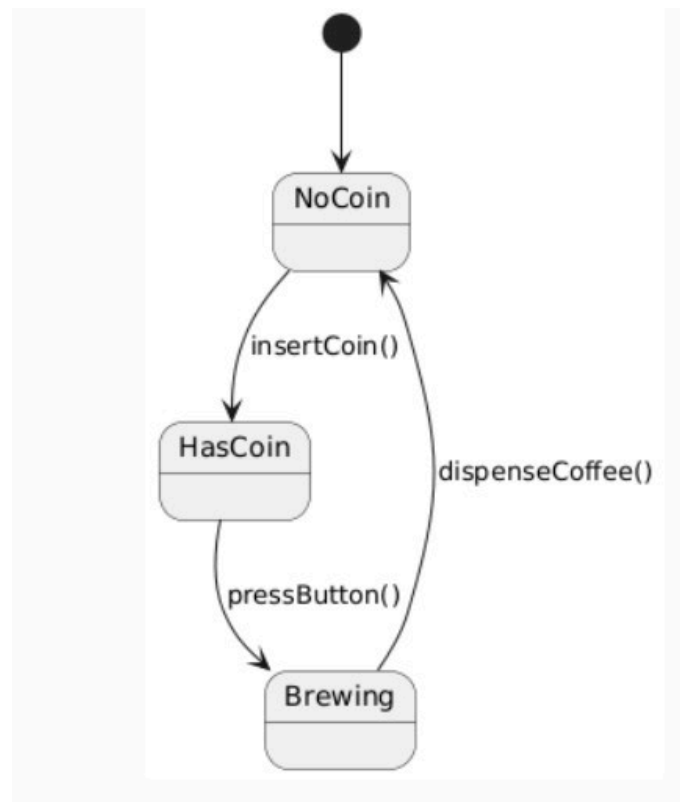
Problema real: Coffe Machine

A máquina de café tem 3 estados:

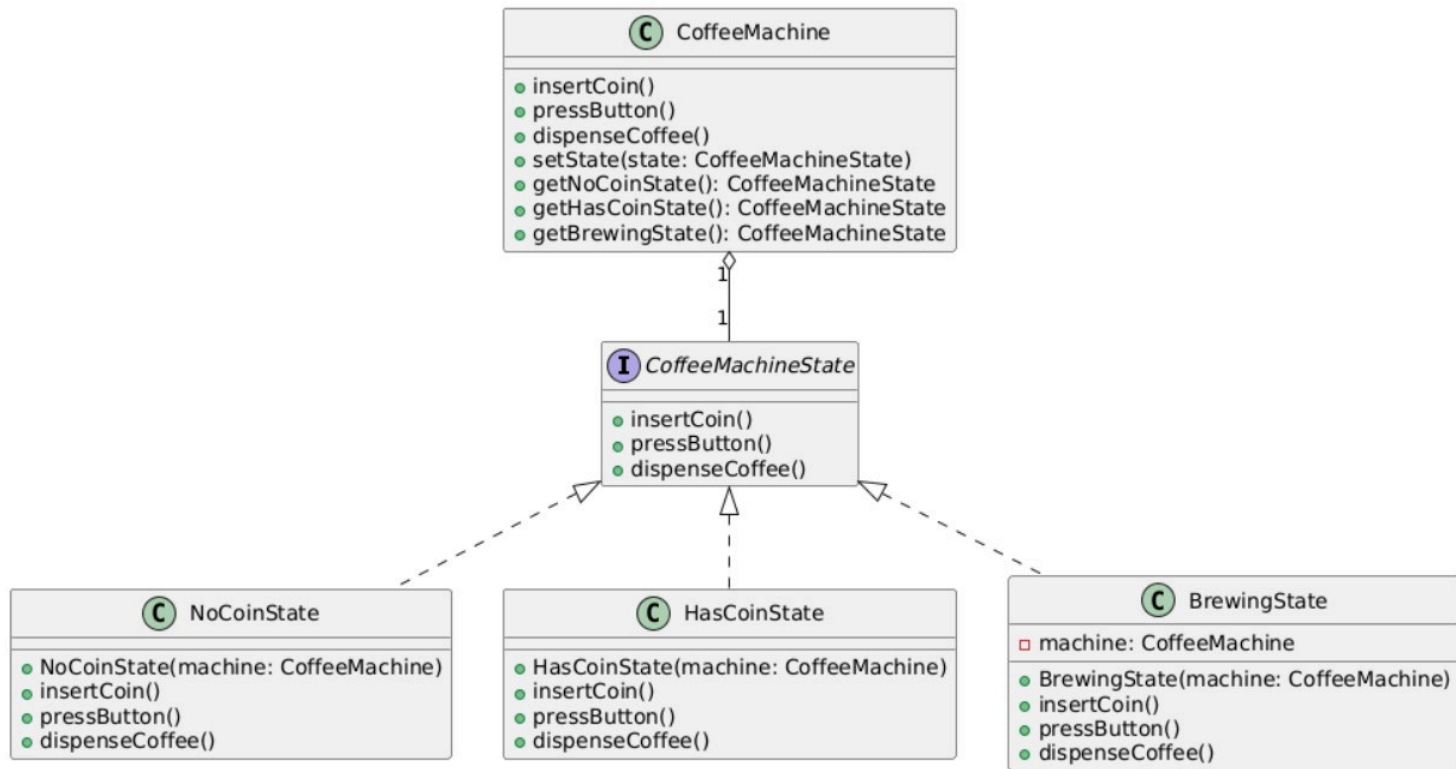
1. **NoCoinState** - Representa a máquina à espera de uma moeda. Aceita a inserção de uma moeda. Rejeita comandos de preparação ou distribuição até que haja pagamento
2. **HasCoinState** - Representa que uma moeda foi inserida e a máquina está pronta para iniciar a preparação. Não aceita moedas adicionais. Permite iniciar o processo de preparação.
3. **BrewingState** - Representa que a máquina está actualmente a preparar o café. Não aceita novas moedas. Não permite pressionar botões adicionais. Distribui o café quando a preparação estiver concluída

Problema real: 🌍 Coffe Machine

A máquina de estados finitos atualmente implementada é a seguinte:



A estrutura de classes que implementa o padrão é:



Atente no código da classe `CoffeMachine` :

- Possui um atributo do tipo `CoffeMachineState` (interface);
- Os estados possíveis atualmente são as concretizações desta interface: `HasCoinState` ou `NoCoinState` ou `Brewing` ;
- O comportamento dos métodos `pressButton` , `insertCoin()` , `dispenseCoffe()` é "delegado" para o *estado* atual.
- O estado inicial (construtor) é o estado `NoCoin` .

Atente ao código do main.

- O código cliente (neste caso, o main) utilize a máquina de café (**CoffeeMachine**) sem precisar conhecer ou manipular diretamente os estados internos (`NoCoinState` , `HasCoinState` , `DispensingState`).

```
public class Main {  
    public static void main(String[] args) {  
        CoffeeMachine machine = new CoffeeMachine();  
        machine.pressButton();  
        machine.insertCoin();  
        machine.pressButton();  
        machine.insertCoin();  
        machine.dispenseCoffee();  
    }  
}
```

Exercício (1/2)

Considere que queremos alterar o funcionamento da máquina de café, para quando devido à falta de stock , esta não pode servir mais café.

Para isto vamos introduzir um novo estado:

OutOfCoffeeState :

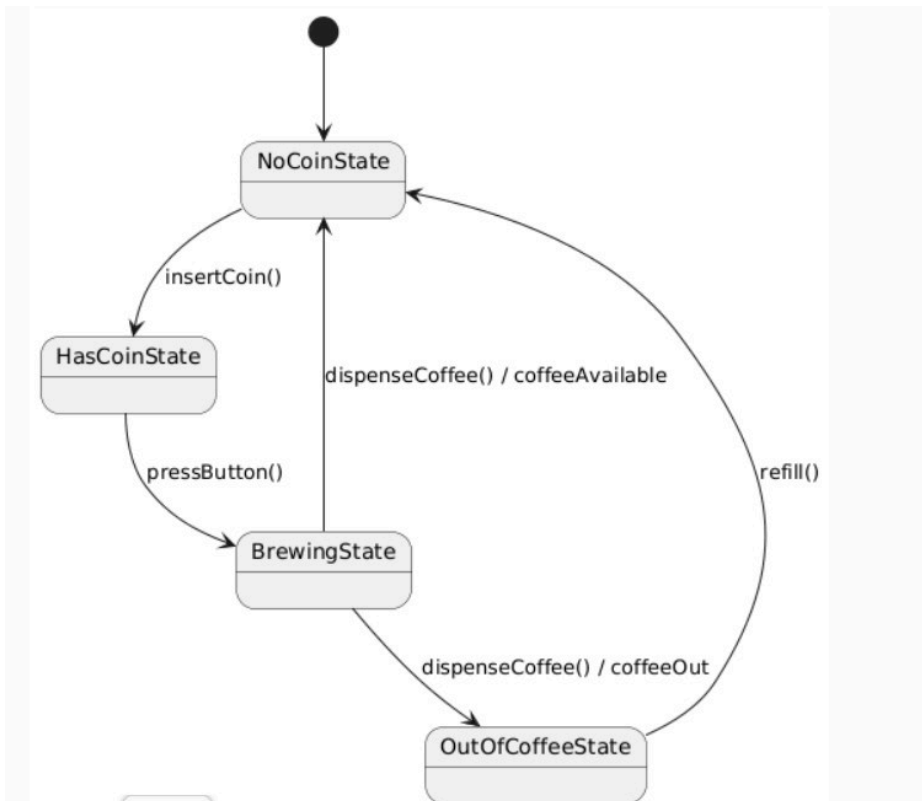
- Não aceita moedas
- Não permite iniciar o processo de preparação
- Informa o utilizador que não há café
- Aguarda o evento refill() para voltar ao estado

NoCoinState

Exercício (2/2)

- 1 - Elabore o novo diagrama de máquina de estados.
- 2 - Implemente a nova classe `OutOfCoffeeState`
- 3 - Modifique as restantes classes para acomodar este novo funcionamento.
- 4 - Atualize o main para ilustrar o funcionamento.

1 - State Diagram



Conclusão: Prós e contras

- ✓ *Single Responsibility Principle*. Organização do código respetivo a cada estado particular em classes separadas.
- ✓ *Open/Closed Principle*. É possível introduzir novos estados com alterações mínimas nas existentes e, por vezes, sem alterar o *contexto*.
- ✓ Simplificar o código do *contexto* eliminando estruturas condicionais difíceis de manter.
- ✗ A aplicação do padrão pode ser um exagero no caso de máquinas de estado muito simples ou que raramente mudam de estado.

Bibliografia

- *Design Patterns: Elements of Reusable Object-Oriented Software* (1994)
- <https://refactoring.guru/design-patterns/state>